

Mathematical Modeling Tutorial

Data Analysis

Dylan He

1 Feature Engineering

As the old saying goes, data and features determine the upper limit of machine learning, and models and algorithms simply approximate that upper limit. Processing the data before analysis is essential and greatly beneficial. Feature engineering mainly has several aspects.

- Data Pre-process
- Feature Selection
- Dimension reduction

I will introduce several algorithms for each aspect, but the experience in feature engineering is valued the most. No "perfect" method is suitable for all situations, but the principle will never change.

1.1 Data Pre-processing

Let's start by processing the original data. When you get unprocessed datasets, there are several questions you may encounter.

- The dimensions and orders of magnitude of the data are different
- Some values are missed
- Qualitative data cannot be applied directly
- Information redundancy
- The dimensionality is too high

Data pre-processing can solve a large part of these questions. Let's solve them one by one.

Standardization and Normalization

Data normalization and standardization are feature scaling methods, which are key steps in data pre-processing. Different evaluation indicators often have different scales and scale units, such a situation will affect the results of data

analysis, to eliminate the influence of the scale between the indicators, data normalization/standardization processing is needed to solve the comparability of data indicators. After the raw data have been normalized/standardized, the indicators are in the same order of magnitude, making them suitable for comprehensive comparative evaluation.

Normalization/standardization is essentially a linear transformation, and linear transformation has many good properties, which determine that changes to the data will not result in "failure", but rather improve the performance of the data, which are the prerequisites for normalization/standardization. For instance, the numerical order will not change after standardization and normalization. Here are their mathematical forms:

$$X_n = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (1)$$

After Normalization, the data will be converted to $[0, 1]$. This normalization method is more applicable in the case of more concentrated values. However, if max and min are unstable, it is easy to make the normalization results unstable, making the subsequent use of the effect also unstable, the actual use of empirical constant values can be used to replace max and min.

$$X_s = \frac{X - \mu}{\sigma} \quad (2)$$

Where μ is the mean value of data, and σ is the standard deviation. This formula is called Z-Score Standardization, which ensures that the transformed data conforms to a standard normal distribution.

Those two methods convert data into different ranges, thus being suitable for different situations. When you are in a situation where:

- Outliers exist
- The data follows a normal distribution

Z-Score Standardization is better. And while

- Range of output is required
- Data is stable and intense

In addition, sometimes the data are greatly divergent, nonlinear transforming is also beneficial, like the Logistic function and hyperbolic tangent function.

Missing Values Processing

There are 3 kinds of values missing. Missing at Random (MAR), Missing Completely at Random (MCAR), and Missing not at Random (MNAR). Each kind corresponds to different strategies. MAR means that the probability of data loss is not related to the lost data itself, but only to the partially observed data. MCAR missing data are completely random, do not depend on any incomplete or complete variables, and do not affect the unbiased nature of the sample.

MNAR means missing data is related to the value of the variable itself. In random cases, the kernel function smoothing is recommended. Common kernel functions are Gaussian kernel and Linear kernel. For the missing values X_m in the data space:

$$X_m = \sum_{i=1}^n e^{\gamma ||X_m - X_n||} \quad (3)$$

X_n is in the neighbor of missing values. In fact, this is a method of interpolation. It ensures the smoothness of data. Kernel functions can be combined with clustering methods, like **KNN**, to find neighbors. You can also directly delete the data with missing values. However, for MNAR cases, this could lead to model bias.

1.2 Feature Selection

Feature selection is an important data preprocessing process. It has two functions. One is to reduce the number of features and reduce the dimensionality to make the model more generalizable and reduce overfitting. The second is to enhance the understanding between features and eigenvalues.

1.2.1 Selection by Norms

Paradigms are functions with the concept of "distance". To make it easier to understand, we can think of paradigms as distances. In mathematics, paradigms include vector paradigms and matrix paradigms. Vector paradigms characterize the size of vectors in a vector space, and matrix paradigms characterize the size of changes caused by matrices. L_p is defined as below:

$$L_p = ||X||_p = \sqrt[p]{\sum_{i=1}^n X_i^p} \quad (4)$$

Here's a classic figure of the relationship between L_p norms and geometric distance. We are interested in L1 and L2 norms. The L1 norm has many names,



Figure 1: Relationship Between Norms and Geometric Distance

such as the familiar Manhattan distance, minimum absolute error, etc. The difference between two vectors can be measured using the L1 norm, such as the

absolute error sum. Mathematically, the L1 norm is called a sparse regular operator. The sparsity of features can be achieved by L1, removing some features that have no information. L2 norm, aka Euclidean Distance, is often used as a regularization term for the optimization objective function to prevent the model from becoming too complex to cater to the training set resulting in overfitting, thus improving the generalization ability of the model.

The mathematical form of supervised machine learning can be expressed as finding parameters w^* such that:

$$w^* = \arg \min \sum \mathcal{L}(y_i, f(x_i, w)) - \lambda \delta(w) \quad (5)$$

where $\mathcal{L}(y_i, f(x_i, w))$ is the error term and $\lambda \delta(w)$ is the normalization term. In the process of learning, you can set $\delta(x)$ as L1/2 a normalization term, and the weights of features without information will become zero automatically.

1.2.2 Filtering Methods

The central idea of filtering is to remove features based on some indicator. Here are common indicators:

- Statistical significance
- Pearson correlation coefficient
- Chi-square verification
- Mutual information and maximal information coefficient (MIC)
- Spearman's ranking correlation coefficient
- Kendall tau coefficient
- SHAP Value

Statistical significance

Hypothesis testing is a statistical method used to make inferences about the population based on sample data. In the context of feature selection, significance testing is used to determine whether there is a significant relationship between a feature and the target variable. The null hypothesis typically assumes that there is no relationship between the feature and the target variable, and the alternative hypothesis assumes that there is a relationship.

The significance level is chosen as a threshold to determine whether the results of the significance test are statistically significant. If the p-value calculated from the test is less than the significance level, we reject the null hypothesis, indicating that there is a significant relationship between the feature and the target variable.

$$t = \frac{\bar{X} - \mu}{s/\sqrt{n}} \quad (6)$$

where t is the test statistic, $\bar{X}$ is the sample mean, μ is the population mean (null hypothesis mean), s is the sample standard deviation, and n is the sample size. We usually choose features with p-values with target variables bigger than 0.05.

Pearson Correlation Coefficient

The Pearson correlation coefficient is a measure of the linear relationship between two continuous variables. It ranges from -1 to 1, where 1 indicates a perfect positive linear relationship, -1 indicates a perfect negative linear relationship, and 0 indicates no linear relationship.

$$r = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum (X_i - \bar{X})^2 \sum (Y_i - \bar{Y})^2}} \quad (7)$$

However, it can only capture the linear relationship. So it doesn't fit all situations.

Chi-square Verification

The chi-squared test, also known as the chi-square test, is a statistical test used to determine whether there is a significant association between two categorical variables. It is based on comparing observed frequencies of data with expected frequencies under the null hypothesis of independence between the variables.

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i} \quad (8)$$

where χ^2 is the chi-squared test statistic, O_i is the observed frequency for category i , and E_i is the expected frequency for category i .

The chi-squared test helps determine whether the differences between the observed and expected frequencies are statistically significant. If the calculated chi-squared value is greater than a critical value from the chi-squared distribution (based on the degrees of freedom and chosen significance level), we reject the null hypothesis of independence and conclude that there is a significant association between the categorical variables.

MIC

Mutual Information and Maximal Information Coefficient (MIC) are both statistical indicators used to measure the degree of association between two variables. Mutual information measures how much information is shared between two random variables.

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right) \quad (9)$$

Unfortunately, it's not convenient to use classic mutual information, because it can not be normalized, which makes it hard to compare between different data sets. It's also not suitable for continuous data, because it requires discretization and it's sensitive to the means of discretization.

The maximal information coefficient can solve those problems. It first searches for an optimal discretization and then converts the mutual information taken

into a metric with values in the interval $[0, 1]$. The calculation of MIC is based on a subsampling variant to ensure robust results even in the presence of duplicate values. MIC is more suitable for detecting nonlinear relationships between variables than linear correlation measures such as the Pearson correlation coefficient. You can call `minepy` for MIC.

Spearman's Rank Correlation Coefficient

The Spearman's rank correlation coefficient, also known as Spearman's rho, is a non-parametric measure of statistical dependence between two variables. Unlike the Pearson correlation coefficient, Spearman's rho assesses the monotonic relationship between variables, which means that it measures how well the relationship between variables can be described using a monotonic function. Spearman's rank correlation coefficient is calculated based on the ranked values of the variables rather than their actual numerical values. The formula for calculating Spearman's rho is as follows:

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (10)$$

Spearman's rank correlation coefficient is often used when the relationship between variables is not linear or when the data contain outliers. It's also helpful when some data is ordinal. For ρ , 1 indicates a perfect positive monotonic relationship, -1 indicates a perfect negative monotonic relationship, and 0 indicates no monotonic relationship between the variables.

Kendall Tau Coefficient The Kendall tau coefficient is also a rank correlation coefficient, but it is calculated for categorical variables. Kendall tau coefficient is based on the counting of (i,j) homoscedastic pairs, where Kendall tau coefficients are based on the counts of (i,j) homoscedastic pairs (where i,j), homoscedastic means that $x_{a,i} - x_{a,j}$ and $x_{b,i} - x_{b,j}$ have the same sign. The Kendall tau equation includes a pairwise correlation coefficient on the normalized constants. includes an adjustment term for the value of the knot in the normalized constant, often referred to as tau-b. For the matrix with column a x_a and column b x_b b columns of the matrix x_a and x_b , the Kendall tau coefficient is defined as:

$$\tau = \frac{2 \sum_{i=1}^{m-1} \sum_{j=i+1}^m \xi}{m(m-1)} \quad (11)$$

The value of the correlation coefficient ranges from -1 to +1. A value of -1

$$\xi = \begin{cases} 1 & \text{if } (x_{a,i} - x_{a,j})(x_{b,i} - x_{b,j}) > 0 \\ 0 & \text{if } (x_{a,i} - x_{a,j})(x_{b,i} - x_{b,j}) = 0 \\ -1 & \text{if } (x_{a,i} - x_{a,j})(x_{b,i} - x_{b,j}) < 0 \end{cases}$$

means that one column order is the inverse of the other column order, and a value of +1 means that the two column orders are the same. A value of -1 means that one column order is the inverse of the other, and a value of +1 means that the two column orders are the same. A value of 0 means that there is no relationship between the columns.

SHAP Value

SHAP (SHapley Additive exPlanations) values are a method used in machine learning to explain the output of a model by assigning each feature an importance value for a particular prediction. SHAP values provide a way to understand the impact of different features on the model's prediction for an individual instance.

$$\phi_i = \frac{1}{M} \sum_{j=1}^M \frac{1}{|S_j|} \sum_{S \subseteq S_j \setminus i} |S|!(|S_j| - |S| - 1)! [f(S \cup i) - f(S)] \quad (12)$$

This is the formula for calculating the approximation of Monte Carlo sampling of Shapley values.

All of those indicators, you can call libraries, like sci-kit learn, minepy, and SciPy.stats for calculating functions.

1.2.3 Gray Correlation Analysis

Gray correlation analysis refers to the method of quantitative description and comparison of the development and change trend of a system, and its basic idea is to determine whether the connection is close or not by determining the degree of geometrical similarity between the reference data column and several comparative data columns, which reflects the degree of correlation between the curves. In the process of system development, if the trend of change of two factors is consistent, i.e., the degree of synchronized change is high, it can be said that the degree of correlation between the two is high; on the contrary, it is low.

Gray correlation analysis method requires that the sample capacity can be as small as 4, which is equally applicable to data irregularity, and there will be no discrepancy between quantitative results and qualitative analysis results.

The calculation process includes the following steps:

- Data normalization: Normalize the raw data to make the data comparable between different indicators.
- Constructing the correlation matrix: calculate the correlation between each factor and construct the correlation matrix.
- Determine the weights: Determine the weights of each factor according to the correlation matrix.
- Finding the comprehensive correlation: Combine the weights and correlations of each factor to get the final comprehensive correlation.
- Discriminate and analyze: according to the size of the comprehensive correlation degree, determine the degree of correlation between the factors, and analyze and explain.

Normalize the data:

$$x_{ij}^* = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)}$$

Where x_{ij}^* is the normalized value of x_{ij} . Calculate the grey relational coefficient:

$$\xi_{ij} = \frac{\min |x_{ki} - x_{kj}| + \rho \max |x_{ki} - x_{kj}|}{\max |x_{ki} - x_{kj}| + \rho \min |x_{ki} - x_{kj}|}$$

Compute the grey relational grade:

$$\rho_i = \frac{1}{n} \sum_{j=1}^n \xi_{ij}$$

1.3 Dimensionality Reduction

Data dimensionality reduction is to reduce the dimensionality of data to achieve the purpose of increasing sample density as well as denoising. According to a certain mathematical transformation method, the data points in the high-dimensional space are mapped into the low-dimensional space, and then the features of the mapped variables are used to represent the original overall features. Here are the common methods.

- Principle Component Analysis (PCA)
- Stochastic Neighbor Embedding (t-SNE)
- Local Linear Embedding (LLE)
- Kernel PCA (KPCA)

1.3.1 Principle Component Analysis

PCA (Principal Component Analysis) is a commonly used dimensionality reduction technique used to map high-dimensional data sets to low-dimensional spaces. Its principle is based on the covariance matrix factorization of data, which realizes dimensionality reduction and feature extraction of data by finding the principal component (i.e., the direction with the largest variance) in the data. The essence of PCA dimensionality reduction is to ensure that the relative position of the data in space remains unchanged as much as possible, and to depict the data by rotating the coordinate system and changing to a coordinate system that can express more data information in certain dimensions. With the PCA algorithm, we can reduce the dimensionality of the data and reduce the number of features, while retaining the most important information in the data. The specific implementation process of PCA algorithm is as follows:

1. Data preprocessing: the raw data are standardized so that the mean of each feature is 0 and the variance is 1 to eliminate the difference in magnitude between different features.

2. Calculation of the covariance matrix: For the standardized data set, the covariance matrix is calculated. The covariance matrix reflects the correlation between individual features in the data.

$$C = \frac{1}{m} X^T X \quad (13)$$

3. Eigenvalue Decomposition: Eigenvalue decomposition is performed on the covariance matrix to obtain the eigenvalues and corresponding eigenvectors. The eigenvalues represent the variance in the data, while the eigenvectors represent the direction of the principal components of the data.

4. Selection of principal components: according to the size of the eigenvalues, select the first k eigenvectors with the largest variance as the principal components set to V_k , where k is the dimensionality after dimensionality reduction, and Z is the final obtained dimensionality reduction matrix.

5. Data transformation: the original data is projected onto the selected principal components to get the dimensionality-reduced dataset.

In MATLAB or Python, you can directly use functions to achieve it.

1.3.2 t-SNE

t-SNE (t-Distributed Stochastic Neighbor Embedding) is a dimensionality reduction technique commonly used for data visualization. It is especially effective at visualizing high-dimensional data in lower-dimensional space while preserving the local structure of the data points. Original SNE computes conditional prob-

Algorithm 1 t-SNE algorithm

```

Compute pairwise similarities  $p_{j|i}$  for high-dimensional data points
Initialize low-dimensional embeddings randomly
Set iteration parameters
for each iteration do
    Compute pairwise affinities  $q_{j|i}$  for low-dimensional embeddings
    Compute gradient  $\frac{\partial C}{\partial y_i}$  for each data point
    Update low-dimensional embeddings using gradient descent
end for

```

abilities in high-dimensional spaces and optimizes the Kullback-Leibler (KL) scattering. By iteratively optimizing the KL scatter, the locations of the samples in the low-dimensional space can be obtained. t-SNE algorithm has the advantage of being able to capture nonlinear data structures while maintaining local similarity between samples. The improvement of t-SNE over SNE is the use of t-distribution to represent the conditional probability in the low-dimensional space to solve the probabilistic sparsity problem in SNE, and the use of symmetric gradient computation in the optimization process to speed up

the convergence.

Conditional possibilities are defined as:

$$P(i|j) = \exp(-||x_i - x_j||^2 / (2 \cdot \sigma^2)) / \Sigma_k \quad (14)$$

Kullback-Leibler Divergence:

$$KL(P|O) = \sum_i \sum_j P(i|j) \log \frac{P(i|j)}{Q(i|j)} \quad (15)$$

1.3.3 Local Linear Embedding

Local Linear Embedding (LLE) is a non-linear dimensionality reduction technique that aims to preserve the local geometry of the data. Unlike linear techniques such as PCA, LLE can capture the intrinsic structure of data that may lie on a non-linear manifold. the LLE algorithm, the first step involves con-

Algorithm 2 Linear Embedding (LLE) algorithm

Construct neighborhood graph using k nearest neighbors
for each data point **do**
 Compute weights to reconstruct data point using neighbors
 Compute local coordinates by solving for optimal reconstruction weights
end for
Compute low-dimensional embeddings by minimizing the cost function

structing a neighborhood graph by connecting each data point to its k nearest neighbors. Then, for each data point, weights are computed to reconstruct the point using its neighbors. By solving for the optimal reconstruction weights, local coordinates are obtained. Finally, the low-dimensional embeddings are computed by minimizing a cost function that ensures the preservation of local relationships in the data.

1.3.4 KPCA

Kernel Principal Component Analysis (KPCA) is a non-linear extension of the classical Principal Component Analysis (PCA) method. It is used for dimensionality reduction and feature extraction in cases where the data is not linearly separable in the original feature space.

In the KPCA algorithm, the first step involves computing a kernel matrix using a chosen kernel function (such as linear, polynomial, or Gaussian kernel). The kernel matrix is then centered to make the data zero-mean in the feature space. Next, the eigendecomposition of the centered kernel matrix is computed to find the eigenvectors and eigenvalues. The top k eigenvectors corresponding to the largest eigenvalues are chosen to form the new feature space, onto which the data is projected to obtain the low-dimensional representation.

Algorithm 3 Kernel Principal Component Analysis (KPCA) algorithm

Compute kernel matrix K using chosen kernel function
Center the kernel matrix K
Compute the eigenvalues and eigenvectors of the centered kernel matrix
Choose the top k eigenvectors corresponding to the largest eigenvalues
Project the data onto the eigenvectors to obtain the low-dimensional representation

2 Regression and Classification

2.1 Dataset Dividing

Typically, we will divide our dataset into 3 parts, the training set, the validation set, and the test set, while we are training our machine learning models. On the training set, we train our models to fit the data. Sometimes we need to examine model configurations and adjust hyper-parameters, then the validation set will be used. The test set will never be used while training. It tests the final effectiveness of our models. If you use the test set to train your models, it will become overfitting and it's kind of cheating to some extent.

A common method of diving is called **K-fold Cross Validation**. Split the original training dataset into K non-overlapping sub-datasets and then do model training and validation K times. Each time, validate the model using one sub-dataset and train the model using other $K-1$ sub-datasets. Finally, average the training error and validation error for each of these K times. For example, as

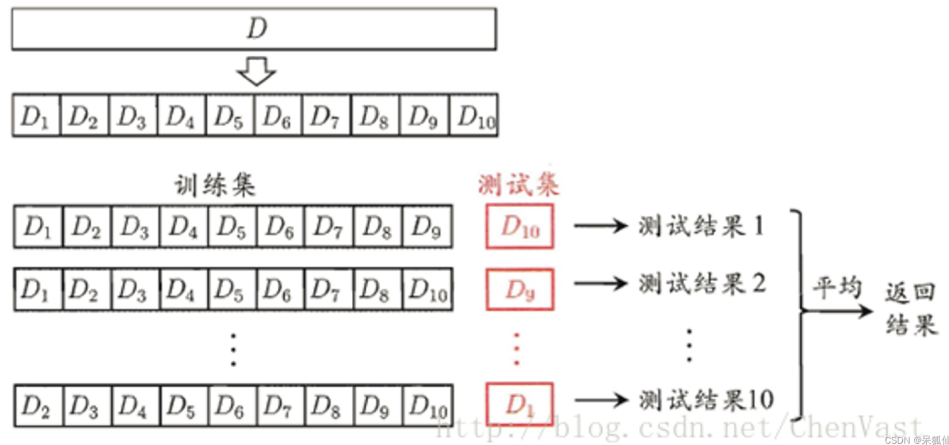


Figure 2: Porcessdure of K-fold Cross Validation

shown in Figure 2 10-fold cross-validation, the data are divided into 10 subsets that do not intersect each other, labeled as D_1, D_2 , and D_{10} , the first time, D_{10} is used as the test set, and the rest is used as the training set to conduct

experiments, and test result 1 is obtained, the second time, D9 is used as the test set, and the rest is used as the training set to conduct experiments, and test result 2 is obtained, and the procedure is repeated 10 times to obtain 10 test results. These 10 test results are averaged to return as the final result.

Training Set	Validation and Test Set	Results
Bad	Bad	Ill-fitting, hyper-parameters adjusting needed
Bad	Good	Your data is improper.
Good	Bad	Overfitting, normalization or dropout needed.
Good	Good	Very Good, you succeeded!

If your models perform well on both training sets and validation sets but badly on test sets, it means that your models are poor in generalization. You can adjust in the same way as overfitting cases.

In the rest of this section, I will introduce many common machine-learning algorithms for regression and classification tasks. They can be roughly divided into several parts: supervised learning, ensemble learning, and deep learning.

2.2 Supervised Learning

Supervised learning is one of the most common and widely used methods in machine learning. It is a method of predicting a target variable from a training dataset in which each training sample has a known label or output value. The presence of labeling is the criterion that determines whether a method is supervised learning or not. To be precise, all regression classification tasks are supervised learning. But when we talk about supervised learning, we tend to refer to the following common machine learning algorithms.

2.2.1 Linear Regression

The classical expression for multiple linear regression is:

$$y = \beta_0 + \sum_{i=1}^n \beta_i x_i \quad (16)$$

Here, the dependent variable y as well as the independent variables $x_1, x_2, \dots, x_n$ are known, and the core of multiple linear regression is to find the multiple regression coefficients β .

$$\beta = \arg \min \sum_{i=1}^n (y^{(n)}(\mathbf{x}, \beta) - y^{*(n)})^2 \quad (17)$$

Take the gradient to the function, we can derive that

$$\vec{\beta} = \mathbf{x}^T y^* (\mathbf{x}^T \mathbf{x})^{-1} \quad (18)$$

Linear regression is the baseline model for regression and classification. Running linear regression as a comparison is necessary whether or not you plan to try a more complex model.

2.2.2 Ridge/LASSO Regression

Ridge regression and LASSO regression are similar, they both add a normalization term to the objective function of linear regression. LASSO regression utilizes L1 normalization while the ridge regression uses L2 normalization. Their loss function can be defined as:

$$P_a(\beta) = \sum_{j=1}^p \left(\frac{1-a}{2} \beta_j^2 + a \beta_j \right) \quad (19)$$

$$\mathcal{L}(\beta) = L_0 + \lambda P_a(\beta) \quad (20)$$

where L_0 is the loss function of linear regression. you can see that if $a = 0$, it's ridge regression. And if $a = 1$, it's LASSO regression. The value of λ allows the model to balance between bias and variance, with the variance of the model decreasing and the bias increasing as λ increases.

Lasso and the ridge regression belong to embedded selection. Embedded feature selection is the integration of the feature selection process with the learner training process, and both of them are done in the same optimization process, i.e., feature selection is performed automatically during the learner training process. L1 normalization can get the sparse solution, thus reducing the weights of features with no information. L2 normalization can reduce the overfitting.

2.2.3 Logistic Regression

Multiple linear regression models require that the dependent variable be a continuous normally distributed variable and that the independent variable be linearly related to the dependent variable. When the dependent variable is a categorical variable and the independent variable is not linearly related to the dependent variable, the conditions for the multiple linear regression model are not met. In this case, the logistic regression model is commonly used to deal with this kind of information. logistic regression analysis belongs to non-linear regression, which is a kind of multiple regression analysis method to study the relationship between the dependent variable for binomial classification or multiple classification results and some influencing factors.

Logistic regression utilizes the **sigmoid** function to compute the possibility.

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (21)$$

The possibility is defined as:

$$P = \frac{\exp \beta_0 + \sum_{i=1}^n \beta_i x_i}{1 + \exp \beta_0 + \sum_{i=1}^n \beta_i x_i} \quad (22)$$

Logistic regression is widely used in binary classification, especially in medical diagnosis. It works well with linearly separable data and is easy to interpret. You can click on the link for further information.

<https://zhuanlan.zhihu.com/p/353112595>

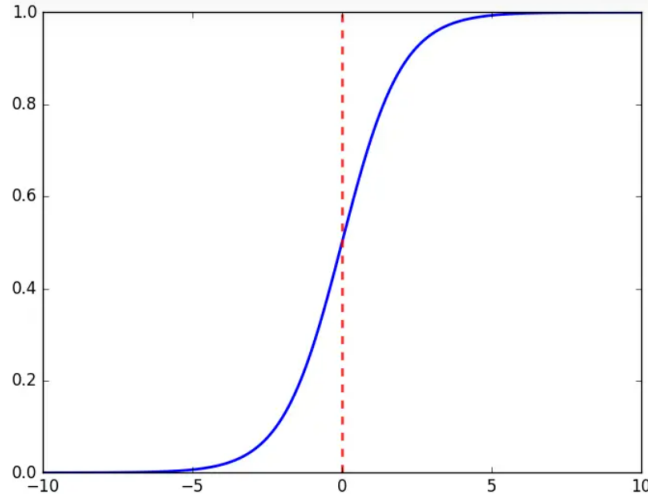


Figure 3: Figure of sigmoid function

2.2.4 Naive Bayesian Classification

Naive Bayes classifiers are based on Bayes' theorem with the "naive" assumption of independence between every pair of features. Despite the oversimplified assumption, Naive Bayes classifiers work well in many real-world situations, especially for text classification and spam filtering.

$$P(y|x_1, x_2, \dots, x_n) = \frac{P(y) \times P(x_1|y) \times P(x_2|y) \times \dots \times P(x_n|y)}{P(x_1) \times P(x_2) \times \dots \times P(x_n)} \quad (23)$$

For a classifier, it's easy to use and has a high computational efficiency. However, it has some drawbacks. It's hard to model dependencies between features and it can be outperformed by more complex models when the independence assumption is violated.

2.2.5 Support Vector Machine

Support vector machines (SVMs) are binary classification models whose basic model is an interval-maximizing linear classifier defined on the feature space. SVMs use a kernel function to map data that is linearly indistinguishable in a low-dimensional space to make it linearly distinguishable in higher dimensions. The learning strategy of SVMs is interval maximization, which can be formalized as a problem of solving a convex quadratic programming problem, and its learning algorithm is an optimization algorithm for solving convex quadratic programming.

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b)) \quad (24)$$

When using a Gaussian kernel, the SVM is a Gaussian radial basis function classifier, in which case the classification decision function is

$$f(x) = \text{sgn}\left(\sum_{i=1}^N a_i x_i \exp -\frac{\|x - z\|^2}{2\sigma^2} + b_i\right) \quad (25)$$

The advantages of SVM classification algorithms include having high accuracy, being able to handle high dimensional data, and performing well for small sample data. However, it also has some drawbacks, such as relative difficulty in handling large-scale datasets and multi-categorization problems, sensitivity to noise and outliers, etc.

2.2.6 Decision Tree

A decision tree is a model that presents decision rules and classification results in a tree-shaped data structure. As an inductive learning algorithm, it focuses on taking known data, which appear to be disordered and cluttered, and transforming them into a tree-like model that can predict the unknown data by some technological means, with each path from the root node (attribute contributing the most to the final classification result) to the leaf node (the final classification result) representing a rule for decision making.

It selects features by two indicators: Information Gain and Gini Impurity. Assume that the data set D was divided into v parts by feature a . Firstly, let's introduce the concept of information entropy. It is a measure of uncertainty in a random variable and indicates the amount of information contained in the variable. The information entropy of original D is:

$$H(D) = - \sum_{i=1}^n P(D_i) \log P(D_i) \quad (26)$$

Then we calculate the information entropy after dividing.

$$H(D^*) = \sum_{j=1}^v \frac{|D_j|}{|D|} H(D_j) \quad (27)$$

Then the Information Gain is:

$$IG(D, a) = H(D) - H(D^*) \quad (28)$$

For all features, the feature with a bigger Information Gain will be considered as an optimal feature. Additionally, the work can also be done by Gini Impurity.

$$Gini(D) = 1 - \sum_{i=1}^n (P(i|D))^2 \quad (29)$$

After choosing the optimal feature at a node, the decision tree will iteratively analyze until all cases in D are defined.

2.3 Ensemble Learning

Ensemble Learning is a machine learning technique that combines multiple individual models (such as decision trees, classifiers, or regressors) to improve the overall performance and predictive power of the model. Instead of relying on a single model's prediction, ensemble learning generates predictions by aggregating the predictions of multiple models. By leveraging the wisdom of the crowd, ensemble learning can often achieve better performance than any individual model. Common ensemble learning methods include Bagging (Bootstrap Aggregating), Boosting, and Stacking.

2.3.1 Random Forest

Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the mode of the classes (classification) or average prediction (regression) of the individual trees. In the process of training, the Bagging Algorithm is used. Traverse the size of the random forest K times, each time with put-back sampling from the training set T . Sample N times to form a new sub-training set D . Randomly select m features, where $m < M$, and use the new training set D and m features to learn a complete decision tree. The use of the Bagging algorithm reduces the overfitting which leads to better

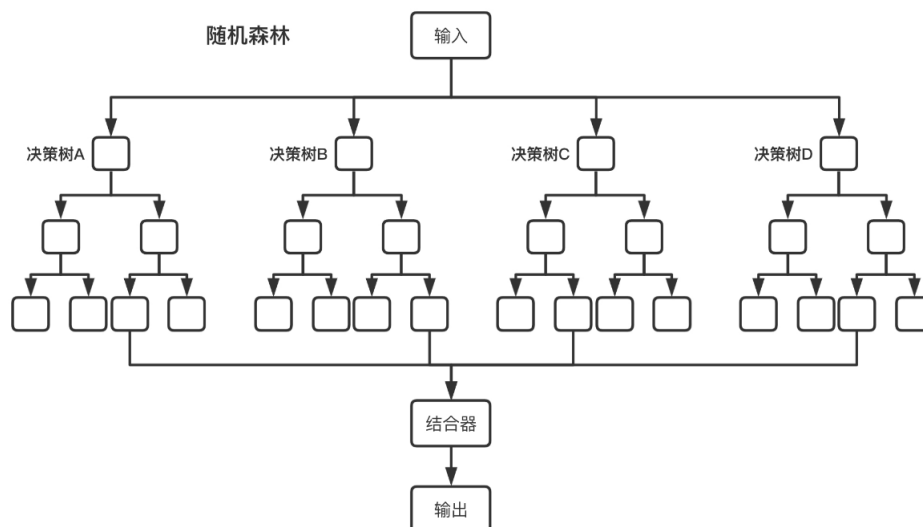


Figure 4: Structure of Random Forest

performance. Individual decision trees are very sensitive to the noise in the training set, but the Bagging algorithm reduces the correlation between multiple decision trees trained, effectively mitigating the above problem. Random forest, as one of the most important ensemble learning methods, has lots of advantages.

- Can produce highly accurate classifiers for many kinds of information.

- A large number of input variables can be handled.
- Can evaluate the importance of variables when deciding on categories
- Can internally produce unbiased estimates of the generalized error when building a forest.
- Includes a good way to estimate missing information and maintain accuracy if a large fraction of the information is missing.
- The error can be balanced for unbalanced sets of categorized data.
- Can be extended to apply to unlabeled profiles, often clustered using unsupervised clustering, and can also detect outliers.
- The learning process is fast.

However, it loses the interpretability of the decision tree.

2.3.2 Gradient Boost Decision Tree

Gradient Boosting Decision Trees is another ensemble technique that builds trees sequentially. Unlike the Bagging or Stacking, it fits new trees to the residuals of the previous tree's predictions. When training the k th decision tree, we minimize the objective function:

$$J = \sum_{i=1}^n \mathcal{L}(y_i, f_{k-1}(x_i) + T_{x_i}) \quad (30)$$

Then we add all outputs from the 1st to the k th decision tree.

$$f_k(x) = \sum_{n=1}^k T_n(x) \quad (31)$$

In a word, the mission of every decision tree is to fit the residual of the last tree. As its name, it uses the gradient descent algorithm to decrease the loss function. However, the hyper-parameters, like the number of weak learners, should be adjusted by hand.

2.3.3 XGBoost

eXtreme Gradient Boost is improved based on GBDT. It adds the normalization term to speed up the training and improve performance. The loss function can be defined as:

$$\mathcal{L}_k = \sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i^{k-1} + f_i(x_i)) + \sum_{k=1}^K \Omega(f_k) \quad (32)$$

Expand the objective function by a second-order Taylor expansion.

$$\mathcal{L}(y_i, \hat{y}_i^{k-1} + f_i(x_i)) \approx \mathcal{L}(y_i, \hat{y}_i) + g_i f_k(x_i) + \frac{h_i}{2} f_k^2(x_i) \quad (33)$$

Combine two equations, we can derive:

$$\mathcal{L}_k = \sum_{i=1}^n [\mathcal{L}(y_i, \hat{y}_i) + g_i f_k(x_i) + \frac{h_i}{2} f_k^2(x_i)] + \sum_{k=1}^K \Omega(f_k) \quad (34)$$

Group the data by leaf nodes and merge the coefficients, we derive:

$$\mathcal{L}^k = \sum_{i=1}^K [G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2] + \gamma T \quad (35)$$

where G_j is the sum of first-order partial derivatives and H_j is the second-order partial derivatives

2.4 Clustering

2.4.1 K-Means Clustering

K-means algorithm, also known as K-means algorithm, belongs to a kind of clustering (clustering) algorithm, and is one of the most widely used clustering algorithms. The so-called clustering, that is, according to the principle of similarity, will have a high degree of similarity of data objects divided into the same class of clusters, and will have a high degree of dissimilarity of data objects divided into different classes of clusters. The biggest difference between clustering and classification is that the clustering process is unsupervised, i.e., the data objects to be processed do not have any a priori knowledge, while the classification process is supervised, i.e., there exists a training dataset with a priori knowledge.

For a given set of samples, divide the sample set into K clusters according to the size of the distance between the samples. Let the points within the clusters be as closely connected as possible, and let the distances between the clusters be as large as possible. Given the sample set D, the k-means algorithm minimizes the squared error for the cluster division C obtained from clustering.

$$E = \sum_{i=1}^k \sum_{x \in C_i} \|x_i - \mu_i\|_2^2 \quad (36)$$

This formula portrays to some extent how closely the samples in the cluster are organized around the cluster mean vector, and the smaller the value of E the higher the similarity of the samples in the cluster. k-means algorithm usually employs L2 norms to compute the distance between data objects. The k-means clustering algorithm step is essentially a model optimization process of the EM algorithm (Expectation-Maximization algorithm (EM)), the specific steps are as follows:

- Randomly select k samples as the mean vector of the initial cluster class
- Divide each sample dataset into clusters that are closest to it

- Update the mean value vector of the cluster class according to the cluster to which each sample belongs;
- Repeat steps (2) (3), when the set number of iterations is reached or the mean value vector of the cluster class is no longer changed, the model construction is completed and the results of the clustering algorithm are output.

The K value has a great impact on the performance. The methods of selection include the Elbow Method and Gap Statistic.

The Elbow method finds the "elbow" of the k-SSE (sum of the squared errors) graph. Gap Statistic is a statistic used to determine the optimal number of

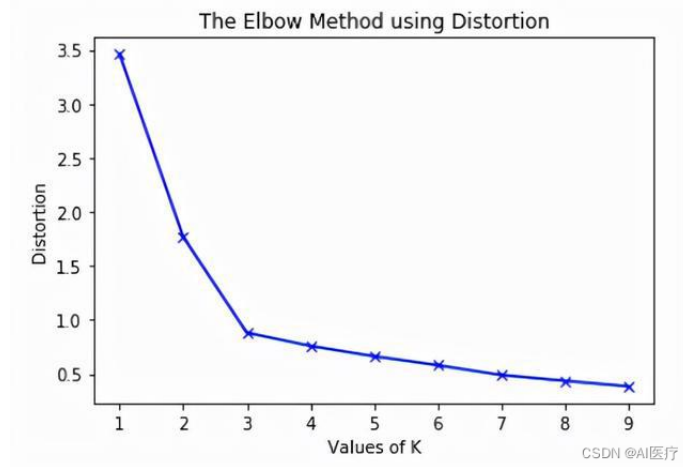


Figure 5: An example of elbow (k=3)

clusters in a data set. It helps to select the optimal number of clusters by determining the appropriateness of the number of clusters by comparing the difference between the actual clustering results of the dataset and the clustering results of a random dataset.

$$Gap(k) = \frac{1}{B} \sum_{b=1}^B \log(W_k^*(b)) - \log(W_k) \quad (37)$$

where W_k is the loss function, and here the previous term refers to the expectation of $\log W_k$. This value is usually generated by Monte Carlo simulation, where we randomly generate as many random samples as the original number of samples in the region where the samples are located according to a uniform distribution and do a K-Means on the random samples to obtain a W_k . Doing this many times, and averaging these values, we get an approximation of expectation. Gap statistic The K that achieves the maximum value is the best K.

2.4.2 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a typical density-based spatial clustering algorithm. Compared to K-Means, and BIRCH, which is generally applicable only to convex sample sets, DBSCAN can be applied to both convex and non-convex sample sets. The algorithm divides regions with sufficient density into clusters and finds arbitrarily shaped clusters in spatial databases with noise, and it defines a cluster as the largest set of densely connected points.

The algorithm utilizes the concept of density-based clustering, which requires that the number of objects (points or other spatial objects) contained in a certain region of the clustered space is not less than a given threshold. The significant advantages of the DBSCAN algorithm are the speed of clustering and the ability to efficiently deal with noisy points and to discover spatial clusters of arbitrary shape. However, when the density of spatial clusters is not uniform and the difference in cluster spacing varies greatly, the quality of clustering is poor.

Algorithm 4 Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

```

1: procedure DBSCAN( $D, \epsilon, \tau$ )
2:    $C \leftarrow 0$ 
3:    $E \leftarrow$  empty list
4:   for  $p \in D$  do
5:      $N(p) \leftarrow$  neighbors of  $p$  within  $\epsilon$  in  $D$ 
6:     if  $|N(p)| < \tau$  then
7:       mark  $p$  as NOISE
8:     else
9:        $C \leftarrow C + 1$ 
10:       $G \leftarrow$  new cluster with  $p$ 
11:       $M \leftarrow N(p)$ 
12:      while  $M$  is not empty do
13:         $q \leftarrow$  arbitrary element from  $M$ 
14:         $M \leftarrow M - q$ 
15:        if  $q$  is not marked as NOISE then
16:          mark  $q$  as part of cluster  $C$ 
17:           $N(q) \leftarrow$  neighbors of  $q$  within  $\epsilon$  in  $D$ 
18:          if  $|N(q)| \geq \tau$  then
19:             $M \leftarrow M \cup N(q)$ 
20:          end if
21:        end if
22:      end while
23:    end if
24:  end for
25: end procedure

```

2.4.3 AMI

AMI (Adjusted Mutual Information) is a metric for evaluating the effectiveness of clustering, which measures the similarity between two clustering results without being affected by variations due to different label naming of clusters. Specifically, AMI combines Mutual Information and Clustering Entropy, and eliminates errors due to chance by adjusting them. AMI takes values in the range of $[0,1]$, with values closer to 1 indicating that two clusters are more similar, and values closer to 0 indicating that the similarity between two clusters is lower. The closer the value is to 1, the more similar the two clusters are, and the closer the value is to 0, the lower the similarity between the two clusters.

$$AMI(Y, C) = \frac{I(Y, C) - E[I(Y, C)]}{\max(H(Y), H(C)) - E[I(Y, C)]} \quad (38)$$

Y denotes the true category label, C denotes the label of the clustering result. $I(Y, C)$ denotes the mutual information of the true category label and the label of the clustering result. $H(Y)$ denotes the entropy of the true category label, $H(C)$ denotes the entropy of the clustering result label. $E[I(Y, C)]$ denotes the expectation of mutual information under random condition.

3 An Example

Here's an example code for your reference. After reading the code, you will find data analysis is quite easy.

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.feature_selection import SelectKBest, f_regression
5 from sklearn.decomposition import PCA
6 from sklearn.linear_model import LinearRegression
7 from sklearn.ensemble import RandomForestRegressor
8 from sklearn.cluster import KMeans
9 from sklearn.metrics import Mean_Squared_Error
10 import matplotlib.pyplot as plt
11
12 # 1. data washing
13 # Assuming we have a file "data.csv" with NaN and ill data.
14 data = pd.read_csv("data.csv")
15 data = data.dropna() # delete missing values
16 data = data[(np.abs(stats.zscore(data)) < 3).all(axis=1)]
17 # drop ill data (z-score>3)
18
19 # 2. Feature selection
20 X = data.iloc[:, :-1]
21 y = data.iloc[:, -1]
```

```

22 selector = SelectKBest(score_func=chi2, k=5)
23 # chi squared test, 5 most relative features
24 X_new = selector.fit_transform(X, y)
25
26 # 3. dimensionality reduction
27 pca = PCA(n_components=2) # reduce to 2-D
28 X_pca = pca.fit_transform(X_new)
29
30 # 4. Linear regression (baseline model)
31 X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.2,
32 random_state=42)
33 reg_1 = LinearRegression().fit(X_train, y_train)
34 y_pred_1 = reg.predict(X_test)
35
36 # 5. Random forest regression
37 reg_2 = RandomForestRegressor().fit(X_train, y_train)
38 y_pred_2 = reg.predict(X_test)
39
40 # Compare the results.
41 rmse_l = np.sqrt(Mean_Squared_Error(y_pred_1, y_test))
42 rmse_rf = np.sqrt(Mean_Squared_Error(y_pred_2, y_test))
43 print("RMSE of linear regression:", rmse_l)
44 print("RMSE of random forest regression:", rmse_rf)
45
46 # 6. clustering
47 kmeans = KMeans(n_clusters=3, random_state=42)
48 kmeans.fit(X_pca)
49 labels = kmeans.labels_
50
51 # 7. visualization
52 plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis')
53 plt.xlabel('Principal Component 1')
54 plt.ylabel('Principal Component 2')
55 plt.title('Clustering Results')
56 plt.show()

```